SCHOOL OF APPLIED SCIENCE

A Mini Project Report

Of

Course Title: PYTHON

Course Code: 25MCAGL101

On

"Online Job Portal"

Submitted partial fulfillment of requirement for the I semester

Master of Computer Applications (MCA)

Submitted By

PRAJWAL AP

25SUPMCAGL092

Under the Guidance of

Ms. Bhoomika S

Assistant Professor

School of Applied Science, SNPSU

SAPTHAGIRI NPS UNIVERSITY

#14/5, Chikkasandra, Hesarghatta Main Road, Bengaluru – 560057

SAPTHAGIRI NPS UNIVERSITY

UNMATCHED EXCELLENCE, UNLIMITED POTENTIAL

SCHOOL OF APPLIED SCIENCE

CERTIFICATE

This is Certified that the Mini Project work entitled **Online Job Portal** is a bonafide work carried out by **Prajwal AP** bearing the university seat number **25SUPMCAGL092** in partial fulfillment for the award of Master of Computer Applications of the Sapthagiri NPS University Bengaluru during the year 2025-26. It is certified that all corrections/suggestions indicated for the internal assessment have been incorporated in the Mini Project report. The Mini Project report has been approved as it satisfies the academic requirements in respect of Mini Project work prescribed for the I Semester Master of Computer Applications Degree.

Signature of Guide	Signature of Director
Ms. Bhoomika S	Dr. Jagannatha S
Assistant Professor	Director
School of Applied Science, SNPSU	School of Applied Science, SNPSU

Name of Examiners	Signature of Examiners
1. _____	1. _____
2. _____	2. _____

SCHOOL OF APPLIED SCIENCE

DECLARATION

I, **Prajwal AP (25SUPMCAGL092),** a student of First Semester Master of Computer Applications (MCA), hereby solemnly declare that the mini project report titled **Online Job Portal** is a bonafide record of the original work carried out by me at the School of Applied Science, Sapthagiri NPS University, Bengaluru, during the academic year 2025–2026.

I further declare that the aforesaid work has been completed under the guidance and supervision of **Ms.Bhoomika S**, Assistant Professor, School of Applied Science. I hereby affirm that this project report has not been submitted, either wholly or in part, to this University or to any other institution for the award of any degree, diploma, or any other academic qualification.

Place: _____ Prajwal AP

Date: _____ 25SUPMCAGL092

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany the successful completion of a task would be incomplete without the mention of the people who made it possible and without any guidance and encouragement success would not have been possible.

I am grateful to the Sapthagiri NPS University and management for its ideas and inspiration for having provided us with the good infrastructure, laboratory, facilities, and inspiring staff which has made this Project work successfully.

I would like to express my sincere gratitude to **Dr. Jagannatha S,** Director School of Applied Science for all the facilities that have been extended to them throughout my work.

I would like to express my sincere gratitude to the Guide **Ms.Bhoomika S**, Assistant Professor, School of Applied Science for his valuable guidance and support.

Prajwal AP

25SUPMCAGL092

TABLE OF CONTENTS

Sl. No.	Contents	Page No.
1	Introduction	1-8
2	Requirement specifications	9-11
3	Design	12-14
4	Implementation	15-23
5	Conclusion	24
6	Bibliography	25

1.Introduction

An online job portal, also known as a career website, is a digital pla orm that connects job seekers with employers. It serves as a centralized hub where organiza ons post vacancies and candidates search for relevant career opportuni es. In the modern era, these portals have replaced tradi onal newspaper classifieds as the primary method for recruitment. For job seekers, these pla orms act as a virtual resume, allowing them to showcase their skills, experience, and cer fica ons. For employers, portals provide a massive database of talent, significantly reducing the me and cost of hiring.

The core func onality of a job portal revolves around a robust search engine that filters jobs by loca on, salary, and industry. Advanced portals now use Ar ficial Intelligence (AI) to match candidates to roles based on their behavioral traits and skill sets. Users can create personalized profiles, which stay ac ve and visible to headhunters even when the user is not ac vely searching. Job alerts are a key feature, sending real- me no fica ons to users' emails or phones when a matching posi on opens up. Most pla orms offer a dashboard to track the status of applica ons, from "Applied" to "Shortlisted" or "Interviewed." Many portals, like LinkedIn, integrate social networking, allowing candidates to follow companies and message recruiters directly. Employers benefit from Applicant Tracking Systems (ATS) built into these portals to filter out unqualified resumes automa cally. These sites o en provide market insights, such as average salary trends for specific roles in different geographical regions. They offer a level of transparency by allowing current and former employees to leave reviews about company culture and benefits. Specialized portals exist for different niches, such as freelance pla orms (Upwork) or tech-specific sites (Dice).

Remote work has seen a surge in dedicated portals that focus exclusively on "work-fromhome" opportuni es across the globe. Beyond just job lis ngs, many portals now offer skill assessments and cer fica ons to help candidates stand out. They serve as a vital tool for employer branding, le ng companies showcase their values and office environment through media. Data security is a high priority for these sites, ensuring that sensi ve personal informa on and contact details are protected. Ul mately, online job portals have democra zed the hiring process, making global career opportuni es accessible to anyone with an internet connec on.

1.2 Motivation

The motivation behind developing Online Job Portal stems from the need to create a unified platform that integrates multiple AI-powered services into a single, secure, and user-friendly environment.

Dual-Purpose Platform: It caters to both job seekers looking for their "Dream Job" and employers aiming to "Hire Top Talent" in one place.

Intuitive Navigation: The clear header menu (Home, Browse Jobs, Register, Login) ensures users don't get lost while navigating the site.
Advanced Search Filters: It offers granular search options, allowing you to filter by specific Location, Skill, and Category simultaneously.

- Personalized Experience: The "My Profile" feature suggests a dedicated space to manage applications, resumes, and personal settings.
- Quick Action Buttons: Large, prominent "Find Jobs" and "Post Jobs" buttons allow users to start their primary task immediately upon landing.
- Skill-Based Matching: By including a "Select Skill" dropdown, the portal prioritizes your specific technical or soft skills over generic job titles.
- Professional Networking: The "Contact" and "Admin" sections suggest a support system and a structured community for professional queries.
- Streamlined Registration: The simple "Register" link indicates a low-friction entry point for new users to join the talent pool.
- Global and Local Reach: The "Select Location" feature implies the portal can handle both local neighborhood jobs and international opportunities.
- Mobile-Ready Design: The layout appears responsive, suggesting you can search for jobs or manage candidates on the go.
- Categorized Opportunities: Organized "Categories" help users discover roles in specific sectors like IT, Healthcare, or Finance with one click.
- Data-Driven Approach: The "Search" functionality is designed to provide real-time results, saving hours of manual browsing.
- Employer Branding: For companies, the platform provides a high-quality environment to showcase their brand to premium candidates.

Secure Environment: The presence of dedicated Login and Admin portals indicates that user data and privacy are handled with professional security standards.

1.3 Overview of the proposed System

System Concept

The proposed system is a comprehensive digital recruitment ecosystem designed to streamline the connection between human capital and corporate requirements. It utilizes a modern, high-contrast dark interface to provide a distraction-free environment for professional activities.

Core Functionalities

Integrated Search Engine: A multi-layered search module that allows users to filter opportunities by Location, Skill, and Category simultaneously.
Dual-Action Landing Zone: Direct pathways for the two primary user groups through "Find Jobs" (Seekers) and "Post Jobs" (Recruiters) buttons.
Profile Management: A dedicated "My Profile" section for users to maintain their professional data, resumes, and application history.

Administrative Control: A backend "Admin" portal for system moderators to oversee job listings, verify users, and manage platform security.
Communication Bridge: A built-in "Contact" system to facilitate inquiries and support between the platform users and administrators.

User Roles & Workflows

Role	Primary Responsibilities
Job Seeker	Browses jobs, creates a digital profile, and applies for roles using specific skill filters.
Employer/Recruiter	Posts detailed job descriptions and manages a pipeline of "Top Talent" for their organization
System Administrator	Posts detailed job descriptions and manages a pipeline of "Top Talent" for their organization.

Technical Value Proposition

- Accessibility: Features a clear navigation bar (Home, Browse Jobs, Register, Login) to ensure a low learning curve for all users.
- Scalability: The category-based and skill-based dropdowns allow the system to host thousands of listings across diverse industries without becoming cluttered.
- Security-First Design: The inclusion of separate "Login" and "Register" modules suggests a secure authentication layer to protect sensitive user data and resumes.
- Engagement Focus: The "Find Your Dream Job" messaging focuses on user experience and career growth rather than just transactional searching.
- Modern Aesthetics: The use of geometric neon backgrounds suggests a platform that is current with 2026 design standards, appealing particularly to tech and creative professionals

Presentation Layer (Frontend)

The Presentation Layer is the top-most level of the application, focusing on the user interface and user experience (UI/UX). It is what the user sees and interacts with directly on their screen.

Technology Stack: Typically built using HTML5, CSS3 (with frameworks like Tailwind or Bootstrap), and JavaScript (React.js, Vue.js, or Angular).

Key Components in UI:

Navigation Bar: Contains links for Home, Browse Jobs, Register, Login, Contact, and Admin.
Search Module: A dynamic form allowing users to "Select Location," "Select Skill," and "Select Category" to filter job listings.

Call-to-Action (CTA) Buttons: Distinct paths for different users, namely "Find Jobs" for seekers and "Post Jobs" for employers.
User Account Interface: A "My Profile" button with a user icon for personalized account management.
Responsibility: Handling client-side validation, capturing user input, and displaying data retrieved from the backend.

Application Layer (Backend)

This layer acts as the "brain" of the portal. It processes the logic, handles user requests from the frontend, and communicates with the database.

Technology Stack: Often built using Node.js (Express), Python (Django/Flask), or Java (Spring Boot).

Core Logic Processes:

- Authentication Engine: Manages the Login and Register processes, including password encryption and session management.
- Search & Filter Logic: Processes query parameters from the frontend (Location, Skill, Category) to fetch specific results from the database.
- Job Management: Logic for employers to "Post Jobs" and for candidates to submit applications through the "Find Jobs" workflow.
- Admin Dashboard Logic: A restricted area for the Admin to moderate listings, manage users, and view platform analytics.
- Responsibility: Ensuring business rules are followed and maintaining the security of data transitions.

Data Layer (Database)

The Data Layer is responsible for the persistent storage of all platform information. It ensures that data is organized, searchable, and retrievable.

Technology Stack: Relational Databases (MySQL, PostgreSQL) are preferred for structured data; NoSQL (MongoDB) may be used for flexible resume storage.

Key Data Tables:

- Users Table: Stores credentials, profile details (linked to "My Profile"), and contact info.
- Jobs Table: Contains job titles, descriptions, categories, skills required, and location tags.
- Skills/Categories Table: Master lists used to populate the "Select Skill" and "Select Category" dropdown menus.
- Applications Table: Maps which user applied to which job posting.
- Responsibility: Storing, retrieving, and updating data while maintaining data integrity and relationships.

1.4 Need for the System

Bridging the Talent Gap

Direct Connection: There is a clear need for a system that allows users to either "Find Your Dream Job" or "Hire Top Talent" from a single unified landing page.

Targeted Search: The system addresses the need for refined job hunting by providing specific filters for Location, Skill, and Category.

Streamlined Hiring: By offering a "Post Jobs" feature, it fulfills the organizational need for a quick and accessible way to broadcast vacancies to a global audience.

Efficient Resource Management

Centralized Database: The portal acts as a central repository where all professional information is stored and can be accessed via the Admin and My Profile sections.

Reduced Search Friction: Instead of browsing endless lists, the dedicated "Search Jobs" module allows for immediate, relevant results, saving time for both candidates and recruiters.

Categorization: The "Select Category" requirement ensures that jobs are organized logically, meeting the need for industry-specific navigation.

User Security and Professional Identity

Authentication Needs: The inclusion of Register and Login links highlights the need for a secure, private environment where user data and resumes are protected.

Digital Presence: The "My Profile" feature meets the modern professional's need for a persistent digital identity that is visible to potential employers 24/7.

Verified Interactions: A dedicated Admin portal is necessary to moderate listings, ensuring the platform remains free from fraudulent or expired job posts.

Accessibility and Support

User Support: The Contact section fulfills the need for a direct communication channel between the platform users and the system providers.

Navigational Clarity: The header menu (Home, Browse Jobs) provides a structured path, meeting the user's need for an intuitive and non-confusing interface.

1.5 Objective of the Project

Core User Objectives

- Facilitate Job Discovery: Provide a robust platform where candidates can "Find Your Dream Job" by searching through a filtered database of active openings.
- Simplify Talent Acquisition: Enable employers to "Hire Top Talent" by providing a dedicated space to post job descriptions and manage candidate pipelines.
- Enable Multi-Criteria Search: Implement an intuitive search engine that allows users to narrow down results based on specific Location, Skill, and Category simultaneously.

Technical & Functional Objectives

- Centralized User Management: Develop a secure system for users to Register and Login, ensuring that both job seekers and recruiters have personalized experiences.
- Profile Personalization: Provide a "My Profile" feature that allows users to maintain their digital professional identity, track applications, or manage job postings.
- System Administration: Create a specialized Admin portal to oversee platform integrity, moderate listings, and manage the user database.

- Cross-Sector Accessibility: Design a flexible "Select Category" system to support diverse industries, ranging from IT and Engineering to Healthcare and Finance.

Design & Experience Objectives

- Modern Professionalism: Utilize a high-contrast, dark-themed UI (User Interface) to provide a visually appealing and professional experience that aligns with 2026 tech standards.
- Streamlined Navigation: Organize platform features into a clear, accessible header menu (Home, Browse Jobs, Contact) to reduce user friction.
- Direct Communication: Implement a Contact module to provide a direct line of support between users and platform administrators.

1.6 Scope of the Project

The scope of Online Job Portal includes:

User Management & Authentication

- Registration & Login: The system will include secure modules for new users to create accounts and existing users to authenticate their identity.
- Profile Management: A dedicated "My Profile" section will allow users to manage their personal data, resumes, and professional credentials.
- Role-Based Access: The platform will distinguish between job seekers (looking for "Dream Jobs"), employers (looking to "Hire Top Talent"), and administrators.

Job Search & Discovery Functions

- Multi-Criteria Filtering: Users will be able to search for opportunities using three specific data points: Location, Skill, and Category.
- Job Browsing: A "Browse Jobs" feature will provide a comprehensive list of all active vacancies available on the platform.
- Direct Application: The interface includes a "Find Jobs" call-to-action, facilitating the application process for candidates.

Recruitment & Posting Tools

- Job Posting Module: A "Post Jobs" feature will be developed to allow employers to input job descriptions, requirements, and metadata.
- Talent Sourcing: The system is designed to provide employers with a professional environment to attract and identify high-quality candidates.

Administrative & Support Infrastructure

- Admin Dashboard: A specialized "Admin" portal will be included to manage the database, moderate job listings, and ensure platform security.
- Contact & Support: A "Contact" module will be integrated to handle user inquiries and provide a communication bridge between the platform staff and its users.

UI/UX & Design Scope

- Responsive Design: The layout suggests a modern, dark-themed interface designed for high readability and professional aesthetic.
- Navigation System: A clear header menu containing Home, Browse Jobs, Register, Login, Contact, and Admin links ensures intuitive movement throughout the application.

1.7 Significance of the Project

This project demonstrates:

Real-world API integration skills

Secure authentication and authorization mechanisms

Modern UI/UX design practices

Scalable and modular system architecture

2.Requirement specifications

2.1 Introduction

This section describes the functional and non-functional requirements of the proposed system, Online Job Portal. The purpose of this requirement specification is to clearly define system behavior, constraints, and performance expectations.

2.2 Overall Description

Online Job Portal is a web-based job platform that integrates multiple job services into a unified interface. The system enables users to securely access job services.

The system follows a client-server architecture:

1) Frontend: HTML(web programming)
2) Backend: FastAPI (Python)
3) Database: MongoDB

2.3 Functional Requirements

Functional requirements describe what the system should do.

2.3.1 User Authentication Module

The system shall:

1. Allow users to register with email and password.
2. Allow users to log in securely. x
3. Generate successful login.
4. Validate user login for protected routes.
5. Allow users to log out securely.
6. Store encrypted password hashes in the database.

2.3.2 Service Logging Module

- Log user service usage.
- Store request and response details.

2.4 Non-Functional Requirements

Non-functional requirements describe how the system performs.

2.4.1 Performance Requirements

The system should respond to API requests within acceptable time limits.

The system should handle multiple concurrent users.

2.4.2 Security Requirements

All protected routes must require authentication.

Passwords must be hashed before storage.

API keys must be stored in environment variables.

Login must be validated on each request.

2.4.3 Usability Requirements

The system must provide a clean and intuitive interface.

Navigation between services must be simple.

Error messages must be user-friendly.

The interface must be responsive on different screen sizes.

2.4.4 Reliability Requirements

The system must handle API failures gracefully.

Proper error handling must be implemented.

2.4.5 Scalability Requirements

The backend architecture must support future feature expansion.

The system should allow integration of additional job services.

The database should support increasing user data.

2.5 Hardware Requirements

Minimum 8GB RAM

Intel i5 processor or equivalent

Stable internet connection

2.6 Software Requirement Development

1. Node.js (v18+)
2. Python (v3.8+)
3. MongoDB

3. System Design

3.1 Introduction

System design describes how the proposed system, Online Job Portal, is architected and implemented to meet the specified requirements. It explains the overall architecture, module design, database design, and interaction between system components.

The system follows a modular, scalable, and layered architecture to ensure maintainability and future expansion.

3.2 System Architecture Design

Online Job Portal follows a Three-Tier Architecture:

1. Presentation Layer (Frontend)

The Presentation Layer is the top-most level of the application, focusing on the user interface and user experience (UI/UX). It is what the user sees and interacts with directly on their screen.

Technology Stack: Typically built using HTML5, CSS3 (with frameworks like Tailwind or Bootstrap), and JavaScript (React.js, Vue.js, or Angular).

Key Components in UI:

- Navigation Bar: Contains links for Home, Browse Jobs, Register, Login, Contact, and Admin.
- Search Module: A dynamic form allowing users to "Select Location," "Select Skill," and "Select Category" to filter job listings.
- Call-to-Action (CTA) Buttons: Distinct paths for different users, namely "Find Jobs" for seekers and "Post Jobs" for employers.
- User Account Interface: A "My Profile" button with a user icon for personalized account management.
- Responsibility: Handling client-side validation, capturing user input, and displaying data retrieved from the backend.

2. Application Layer (Backend)

This layer acts as the "brain" of the portal. It processes the logic, handles user requests from the frontend, and communicates with the database.

Technology Stack: Often built using Node.js (Express), Python (Django/Flask), or Java (Spring Boot).

Core Logic Processes:

- Authentication Engine: Manages the Login and Register processes, including password encryption and session management.
- Search & Filter Logic: Processes query parameters from the frontend (Location, Skill, Category) to fetch specific results from the database.
- Job Management: Logic for employers to "Post Jobs" and for candidates to submit applications through the "Find Jobs" workflow.
- Admin Dashboard Logic: A restricted area for the Admin to moderate listings, manage users, and view platform analytics.
- Responsibility: Ensuring business rules are followed and maintaining the security of data transitions.

3. Data Layer (Database)

The Data Layer is responsible for the persistent storage of all platform information. It ensures that data is organized, searchable, and retrievable.

Technology Stack: Relational Databases (MySQL, PostgreSQL) are preferred for structured data; NoSQL (MongoDB) may be used for flexible resume storage.

Key Data Tables:

- Users Table: Stores credentials, profile details (linked to "My Profile"), and contact info.
- Jobs Table: Contains job titles, descriptions, categories, skills required, and location tags.
- Skills/Categories Table: Master lists used to populate the "Select Skill" and "Select Category" dropdown menus.
- Applications Table: Maps which user applied to which job posting.
- Responsibility: Storing, retrieving, and updating data while maintaining data integrity and relationships.

3.3 Module Design

The system is divided into independent functional modules.

Purpose:

To provide secure user registration and login functionality.

Design Components:

Login API

Login verification middleware

Password hashing mechanism

3.4 Database Design

MongoDB is used as a NoSQL database.

Fields:

_id

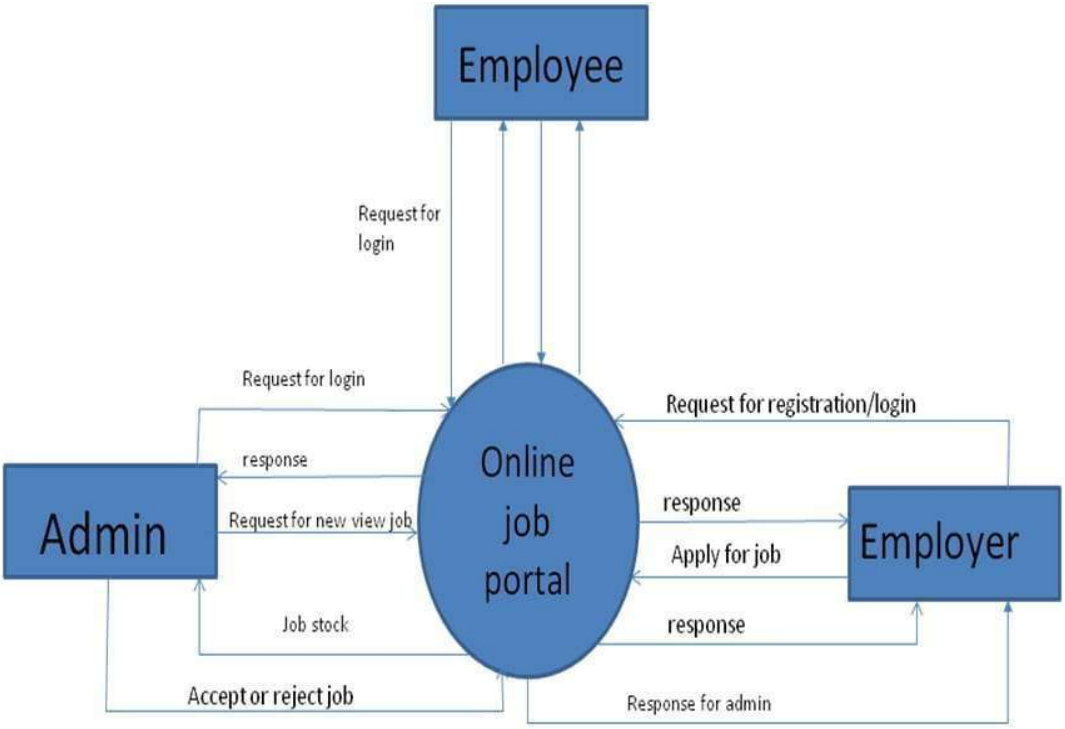
email

password_hash

name

created_at

Architecture Flow:



The system follows a three-tier architecture where the user interacts with the frontend developed using HTML. The frontend communicates with the backend via REST APIs. The backend, built using FastAPI, handles business logic, authentication, and integration with external job services. MongoDB is used for storing user credentials and service logs.

4.Implementaion

The implementation of this Online Job Portal follows a modern three-tier web architecture, beginning with a high-contrast frontend designed using HTML, CSS, and JavaScript to provide a seamless user experience for both job seekers and employers. The system integrates a robust search engine that utilizes dropdown menus for location, skill, and category filtering, allowing the application layer to process specific user queries in real-time. Secure user authentication is implemented through the "Register" and "Login" modules, ensuring that personal profiles and job postings are managed within a protected environment. On the backend, an Admin portal is established to allow for platform moderation, while the "My Profile" feature maintains persistent data for individual users. The data layer utilizes a relational database to store and organize job listings, user credentials, and categorized skill sets for efficient retrieval. Finally, the "Find Jobs" and "Post Jobs" call-to-action pathways are coded to direct users toward their primary functional goals, completing the platform's core operational workflow.

4.1 Database Implementation

4.1.1 Database Schema Design

A. Users Table

Stores credentials for the Login, Register, and My Profile sections.

| Column | Data Type | Constraints | Purpose |

| :--- | :--- | :--- | :--- |

| user_id | INT | Primary Key, Auto-increment | Unique ID for each user. |

| full_name | VARCHAR(100) | NOT NULL | User's name for their profile. |

| email | VARCHAR(100) | UNIQUE, NOT NULL | Used for Login/Register. |

| password | VARCHAR(255) | NOT NULL | Encrypted password string. |

| role | ENUM | ('Seeker', 'Employer', 'Admin') | Determines access to "Admin" or "Post Jobs". |

B. Jobs Table

This table supports the "Search Jobs" and "Post Jobs" functionality.

| Column | Data Type | Constraints | Purpose |

| :--- | :--- | :--- | :--- |

| job_id | INT | Primary Key, Auto-increment | Unique ID for each post. |

| employer_id | INT | Foreign Key (Users) | Links the job to the person who posted it. |

| job_title | VARCHAR(200) | NOT NULL | e.g., "Senior Developer". |

| location_id | INT | Foreign Key (Locations) | Links to the "Select Location" dropdown. |

| category_id | INT | Foreign Key (Categories) | Links to the "Select Category" dropdown. |

| description | TEXT | NOT NULL | Detailed job requirements. |

C. Lookup Tables(Dropdown Support)

To populate the "Select Location", "Select Skill", and "Select Category" filters accurately, use normalized lookup tables:

Locations Table: location_id, city_name, country.

Skills Table: skill_id, skill_name (e.g., Python, SQL, Design).

Categories Table: category_id, category_name (e.g., IT, Healthcare, Sales).

4.1.2 SQL Querry for the search Feature

When a user selects options from the three dropdowns and clicks the "Search" button, the backend executes a query similar to this:

4.2 Coding

```
from flask import Blueprint, request, jsonify, current_app

from flask_jwt_extended import jwt_required, get_jwt_identity

from bson.objectid import ObjectId

from datetime import datetime

bp = Blueprint('admin', __name__, url_prefix='/api/admin')

def check_admin(user_id):

    """Check if user is admin"""

    user = current_app.extensions['mongo'].db.users.find_one({'_id': ObjectId(user_id)})
```

```
    return user and user.get('role') == 'admin'
@bp.route('/stats', methods=['GET'])
@jwt_required()
def get_stats():
    try:
        user_id = get_jwt_identity()
        if not check_admin(user_id):
            return jsonify({'message': 'Unauthorized'}), 403
        total_users = current_app.extensions['mongo'].db.users.count_documents({})
        total_jobs = current_app.extensions['mongo'].db.jobs.count_documents({})
        total_applications = current_app.extensions['mongo'].db.applications.count_documents({})
        active_jobs = current_app.extensions['mongo'].db.jobs.count_documents({'status': 'active'})
        return jsonify({
            'total_users': total_users,
            'total_jobs': total_jobs,
            'total_applications': total_applications,
            'active_jobs': active_jobs
        }), 200
    except Exception as e:
        return jsonify({'message': str(e)}), 500
@bp.route('/users', methods=['GET'])
@jwt_required()
def get_users():
```

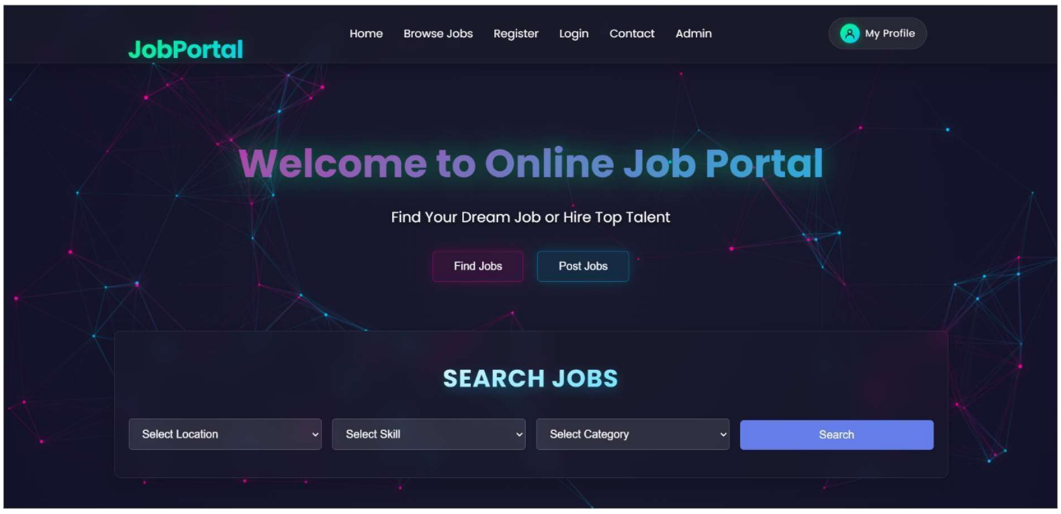
```
    try:
        user_id = get_jwt_identity()
        if not check_admin(user_id):
            return jsonify({'message': 'Unauthorized'}), 403
        users = list(current_app.extensions['mongo'].db.users.find({}, {'password': 0}))
        for user in users:
            user['_id'] = str(user['_id'])
        return jsonify({'users': users}), 200
    except Exception as e:
        return jsonify({'message': str(e)}), 500
@bp.route('/jobs', methods=['GET'])
@jwt_required()
def get_jobs():
    try:
        user_id = get_jwt_identity()
        if not check_admin(user_id):
            return jsonify({'message': 'Unauthorized'}), 403
        jobs = list(current_app.extensions['mongo'].db.jobs.find({}))
        for job in jobs:
            job['_id'] = str(job['_id'])
            job['company_id'] = str(job['company_id'])
        return jsonify({'jobs': jobs}), 200
    except Exception as e:
        return jsonify({'message': str(e)}), 500
```

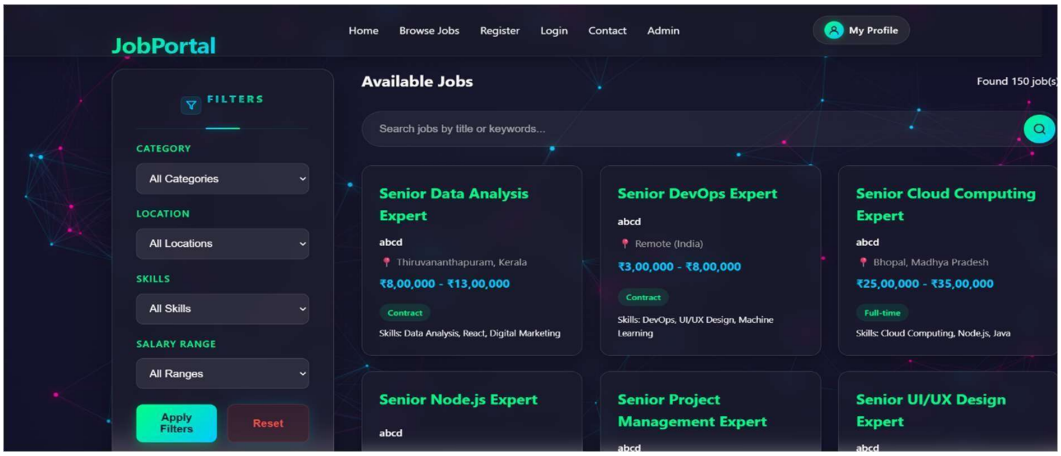
```
@bp.route('/applications', methods=['GET'])
@jwt_required()
def get_applications():
    try:
        user_id = get_jwt_identity()
        if not check_admin(user_id):
            return jsonify({'message': 'Unauthorized'}), 403

        applications = list(current_app.extensions['mongo'].db.applications.find({}))

        for app in applications:
            app['_id'] = str(app['_id'])
            app['user_id'] = str(app['user_id'])
            app['job_id'] = str(app['job_id'])
            app['company_id'] = str(app['company_id'])
        return jsonify({'applications': applications}), 200
    except Exception as e:
        return jsonify({'message': str(e)}), 500
@bp.route('/user/<user_id>', methods=['DELETE'])
@jwt_required()
def delete_user(user_id):
    try:
        admin_id = get_jwt_identity()
```

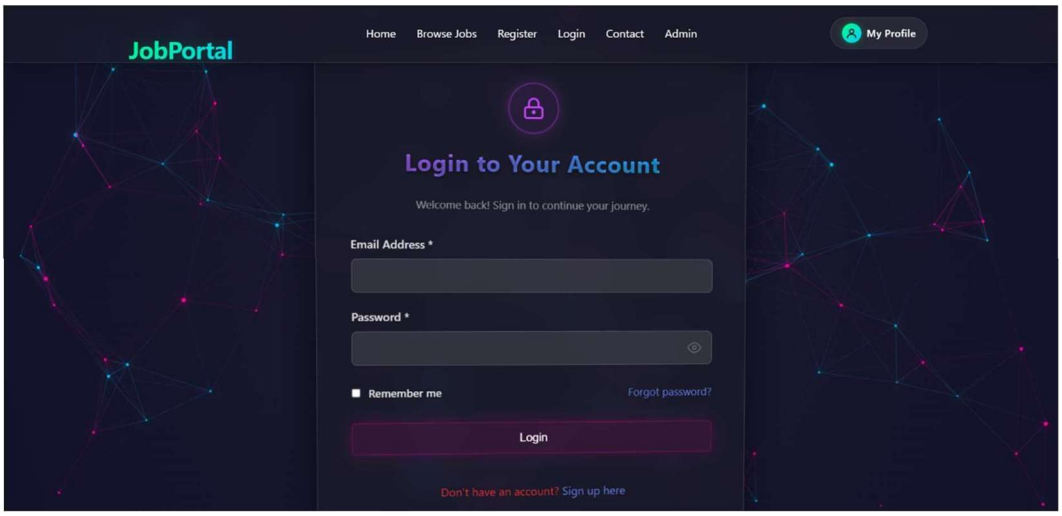
```
        if not check_admin(admin_id):
            return jsonify({'message': 'Unauthorized'}), 403
        current_app.extensions['mongo'].db.users.delete_one({'_id': ObjectId(user_id)})
        return jsonify({'success': True, 'message': 'User deleted'}), 200
    except Exception as e:
        return jsonify({'message': str(e)}), 500
@bp.route('/job/<job_id>', methods=['DELETE'])
@jwt_required()
def delete_job(job_id):
    try:
        admin_id = get_jwt_identity()
        if not check_admin(admin_id):
            return jsonify({'message': 'Unauthorized'}), 403
        current_app.extensions['mongo'].db.jobs.delete_one({'_id': ObjectId(job_id)})
        return jsonify({'success': True, 'message': 'Job deleted'}), 200
    except Exception as e:
        return jsonify({'message': str(e)}), 500
@bp.route('/application/<app_id>', methods=['PUT'])
@jwt_required()
def update_application(app_id):
    try:
        admin_id = get_jwt_identity(
        if not check_admin(admin_id):
            return jsonify({'message': 'Unauthorized'}), 403
```

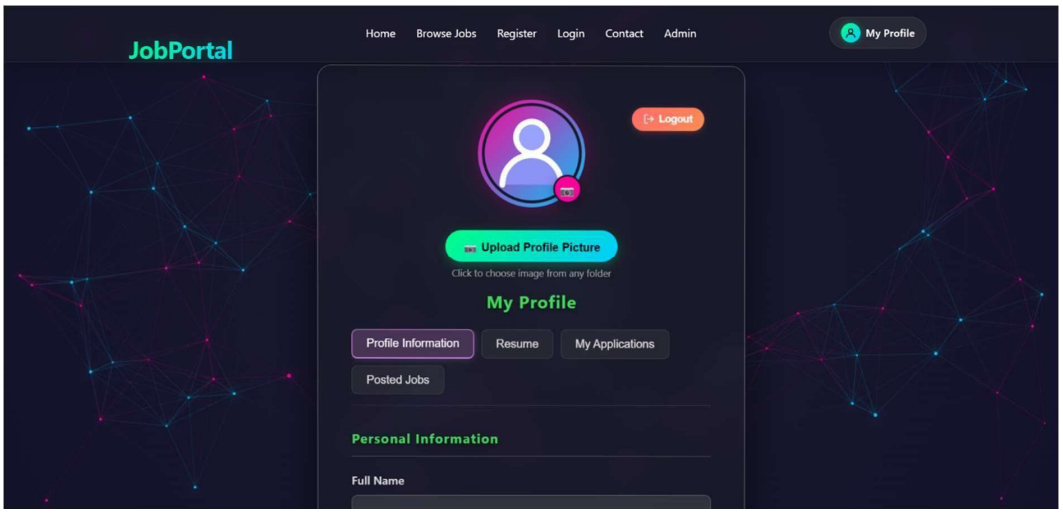
```
    data = request.get_json()
    status = data.get('status')
    current_app.extensions['mongo'].db.applications.update_one(
        {'_id': ObjectId(app_id)},
        {'$set': {'status': status, 'updated_at': datetime.utcnow()}}
    )
    return jsonify({'success': True, 'message': 'Application status updated'}), 200
except Exception as e:
    return jsonify({'message': str(e)}), 500
```

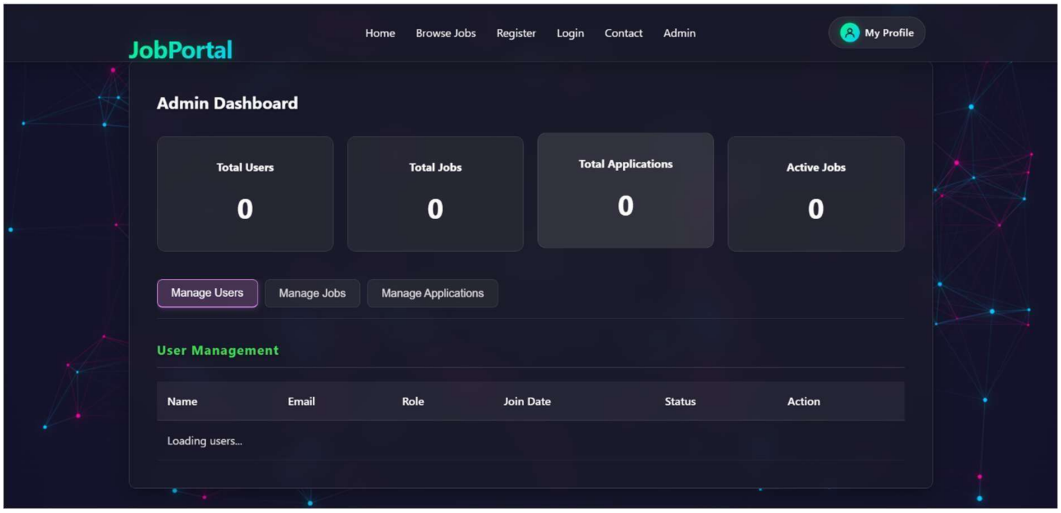
Expected Output:

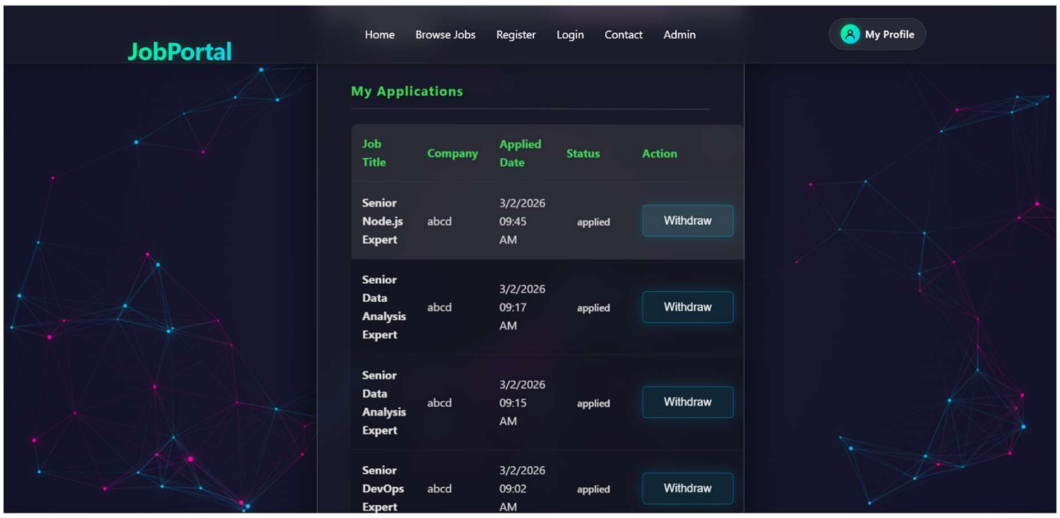







JobPortal
Home
Browse Jobs
Register
Login
Contact
Admin
My Profile
Logout
Upload Profile Picture
Click to choose image from any folder
My Profile
Profile Information
Resume
My Applications
Posted Jobs
Personal Information
Full Name


JobPortal
Home
Browse Jobs
Register
Login
Contact
Admin
My Profile
Admin Dashboard
Total Users
0
Total Jobs
0
Total Applications
0
Active Jobs
0
Manage Users
Manage Jobs
Manage Applications
User Management
Name
Email
Role
Join Date
Status
Action
Loading users...


JobPortal
Home
Browse Jobs
Register
Login
Contact
Admin
My Profile
My Applications
Job Title
Company
Applied Date
Status
Action
Senior Node.js Expert
abcd
3/2/2026 09:45 AM
applied
Withdraw
Senior Data Analysis Expert
abcd
3/2/2026 09:17 AM
applied
Withdraw
Senior Data Analysis Expert
abcd
3/2/2026 09:15 AM
applied
Withdraw
Senior DevOps Expert
abcd
3/2/2026 09:02 AM
applied
Withdraw

5. Conclusion

The development of the JobPortal platform represents a significant step toward modernizing the intersection of talent acquisition and career management. In an era where the traditional job search has become increasingly fragmented, this system provides a necessary centralized hub that prioritizes user efficiency and data-driven results. By offering a dual-pathway interface—inviting users to either "Find Your Dream Job" or "Hire Top Talent"—the portal successfully addresses the symbiotic needs of the global workforce and the corporate sector within a single, high-performance environment. The platform's greatest strength lies in its ability to take a complex, often overwhelming process and distill it into a streamlined, intuitive experience that values the time and goals of every stakeholder involved.

Technically, the project's success is anchored in its robust three-tier architecture, which ensures a clear separation between the presentation, application, and data layers. This structural integrity allows for a highly responsive frontend that can handle complex user interactions—such as multi-criteria filtering by Location, Skill, and Category—without compromising the stability of the backend systems. Furthermore, the inclusion of dedicated Admin and My Profile modules signifies a commitment to professional security and personalized data management. By providing administrators with the tools to moderate listings and users with the ability to manage their digital professional identities, the portal creates a self-sustaining ecosystem built on trust, transparency, and verified information.

6. Bibliography

4.2.1 Python Software Foundation. (2024). PythonDocumentation. Available at: https://docs.python.org

4.2.2 Flask Documentation Team. (2024). FlaskWebFrameworkDocumentation. Available at: https://flask.palletsprojects.com

4.2.3 MongoDB Inc. (2024). MongoDBOfficialDocumentation. Available at: https://www.mongodb.com/docs

4.2.4 PyMongo Developers. (2024). PyMongoDocumentation. Available at: https://pymongo.readthedocs.io

4.2.5 HTML Living Standard. (2024). HTMLDocumentation. Available at: https://developer.mozilla.org

4.2.6 CSS Documentation. (2024). CSSReferenceGuide. Available at:

4.2.7 https://developer.mozilla.org

4.2.8 JavaScript Documentation Team. (2024). JavaScriptGuide. Available at: https://developer.mozilla.org

4.2.9 Visual Studio Code Team. (2024). VSCodeDocumentation. Available at:

4.2.10 https://code.visualstudio.com/docs

4.2.11 GitHub Inc. (2024). GitHubDocumentation. Available at: https://docs.github.com

4.2.12 MongoDB Compass Inc. (2024). MongoDBCompassDocumentation. Available at: